

DSpark: Confidence-Scheduled Speculative Decoding with

Semi-Autoregressive Generation

DSpark: 基于置信度调度的半自回归推测解码

Xin Cheng^{1,2,*}, Xingkai Yu^{2,*}, Chenze Shao^{2,*}, Jiashi Li^{2,*}, Yunfan Xiong^{2,*},
Yi Qian², Jiaqi Zhu², Shirong Ma², Xiaokang Zhang², Jiasheng Ye², Qinyu Chen²,
Chengqi Deng², Jiping Yu², Damai Dai², Zhengyan Zhang², Yixuan Wei², Yixuan Tan²,
Wenkai Yang², Runxin Xu², Yu Wu², Zhean Xu², Xuanyu Wang², Muyang Chen²,
Rui Tian², Xiao Bi², Zhewen Hao², Shaoyuan Chen², Huanqi Cao², Wentao Zhang²,
Anyi Xu², Huishuai Zhang¹, Dongyan Zhao¹, Wenfeng Liang²
1北京大学
2DeepSeek-AI
{chengxin, xingkai, shaochenze, js.li, yunfanxiong}@deepseek.com

摘要

推测解码通过将草稿生成与目标验证解耦，加速了大语言模型（LLM）的推理。虽然近期的并行草稿模型能够在单次前向传播中高效地生成词元序列，但由于缺乏词元间的依赖关系，它们会遭受快速的接收衰减。此外，不加区分地验证这些扩展块会将关键的批处理容量浪费在具有高拒绝风险的词元上，严重降低了高并发服务系统中的吞吐量。我们提出了DSpark，一个统一了高吞吐量并行生成与自适应、负载感知验证的推测解码框架。为了保持草稿质量，DSpark采用了一种半自回归架构——将并行主干与轻量级顺序模块相结合——以引入块内依赖建模并缓解后缀衰减。为了优化系统效率，DSpark采用了基于置信度调度的验证，根据估计的前缀存活概率和引擎特定的吞吐量配置文件，为每个请求动态定制验证长度。在跨不同领域的离线基准测试中，DSpark在接收长度上显著超越了最先进的自回归和并行草稿模型。当部署在DeepSeek-V4服务系统中、处于真实用户流量下时，DSpark成功缓解了验证浪费。与已建立的生产基线（MTP-1）相比，DSpark在匹配的吞吐量水平下将每用户生成速度提升了60% - 85%。更重要的是，通过防止在严格交互性约束下出现严重的吞吐量退化，它实现了以前无法达到的性能层级，改变了我们服务系统的帕累托前沿。为了促进社区进步，我们开源了DSpark检查点以及DeepSpec，一个算法驱动的推测解码训练仓库。

1. 引言

大语言模型（LLM）以自回归方式生成文本：每个新词元都需要以所有先前词元为条件进行一次完整的前向传播，这使得推理延迟与输出长度成正比。由此导致的低GPU利用率和长用户感知等待时间构成了生产级LLM服务的首要瓶颈，特别是对于延迟敏感场景，如实时对话助手和多轮智能体工作流。推测解码（Chen et al., 2023; Leviathan et al., 2023）提供了一种原则性的解决方案：一个轻量级草稿模型提出一个候选词元块，全尺寸的目标模型通过拒绝采样在单次前向传播中验证整个块，接受与目标分布一致的最长前缀并附加一个奖励词元。由于验证是并行的，并且接受规则精确保持了目标分布，推测解码在没有任何质量损失的情况下加速了生成。

草稿模型的设计决定了草稿延迟与接收率之间的权衡。早期的草稿模型是自回归的（Cheng et al., 2024; Li et al., 2024b），每个位置以先前采样的词元为条件。然而，它们的草稿延迟随块大小线性增长，迫使这些方法使用短块和浅层架构。为了打破这种顺序瓶颈，并行草稿模型（Cai et al., 2024; Chen et al., 2026; Liu et al.,

2026a）已成为一种有吸引力的替代方案：所有草稿位置在单次前向传播中生成，使得草稿延迟几乎与块大小无关。这种结构优势理论上允许并行草稿模型高效地生成更长的草稿块。

然而，充分释放大并行草稿块的潜力引入了两个关键瓶颈——一个在生成质量上，另一个在系统效率上。首先，由于并行草稿模型独立地预测每个位置，它们无法建模块内词元间的依赖关系。这种独立性导致多模态碰撞以及后续位置的快速接收衰减（Gu et al., 2018; Huang et al., 2022b）。其次，确定最优的验证长度仍然是一个挑战。虽然并行生成容易产生长草稿块，但不加区分地验证所有提议的词元会降低系统吞吐量，特别是在高并发工作负载下（Hu et al., 2026; Liu et al., 2024c）。理想的验证长度沿两个轴变化。在数据方面，像代码这样的结构化请求自然比开放性聊天具有更高的接收率（Abramovich et al., 2026; Xia et al.,

2024)。在系统方面，在轻负载下验证额外词元几乎免费。然而，在高负载下，验证具有高拒绝风险的词元占用了本可以服务于其他活跃请求的关键批处理容量（Liu et al., 2024b; Wu et al., 2025）。

为了解决这些瓶颈，我们提出了DSpark，一个统一了高吞吐量并行生成与自适应、负载感知验证的推测解码框架。其核心设计旨在通过两种互补机制解决草稿生成和验证中固有的权衡。

•

首先，为了克服缺乏词元间依赖关系的问题，DSpark采用了一种半自回归架构。它保持计算昂贵的草稿主干完全并行，仅附加一个轻量级顺序输出头来注入局部转移信息。这种设计保留了并行模型的草稿速度，同时显著缓解了后缀衰减。

•

其次，为了解决系统级瓶颈，DSpark采用了基于置信度调度的验证。通过将估计每个位置前缀存活概率的置信度头与一个硬件感知调度器相结合，DSpark为每个请求动态定制验证长度。该调度器利用实时引擎吞吐量配置文件，仅将目标验证预算分配给具有最高预期回报的词元。

我们在受控的离线基准测试和生产级在线部署中对DSpark进行了广泛评估。在受控的离线基准测试——涵盖数学推理、代码生成和日常聊天——中，DSpark持续优于强基线。具体来说，在Qwen3-4B、8B和14B目标模型（Yang et al., 2025）上，与自回归的Eagle3（Li et al., 2026b）相比，它分别将宏观平均接收长度提高了30.9%、26.7%和30.0%；与并行的DFlash（Chen et al., 2026）相比，分别提高了16.3%、18.4%和18.3%。除了顶级指标外，我们细粒度的逐位置分析揭示了不同草稿模型的独特生成特征，实验证明了DSpark如何成功地将并行模型的高初始词元能力与自回归模型的后缀连贯性相结合。

除了离线评估，我们将DSpark部署在DeepSeek-V4（DeepSeek-AI, 2026）服务系统中，以评估其在真实用户流量下的性能。与之前的MTP-1生产基线（DeepSeek-AI, 2024）相比，DSpark显著拓宽了系统的运行包络。具体来说，在匹配的总吞吐量能力下，它持续将每用户生成速度提高60%–85%（V4-Flash）和57%–78%（V4-Pro）。此外，在严格的服务水平协议（SLA）下，当基线的容量严重恶化时——例如Flash为120 TPS，Pro为50 TPS——DSpark缓解了验证开销以维持稳健的吞吐量。通过克服这一性能悬崖，DSpark解锁了以前无法达到的严格交互性层级，有效地改变了LLM服务的帕累托前沿。

为了促进开源社区的集体进步，我们公开提供我们的工件。具体来说，我们发布了针对DeepSeek-V4-Flash（预览版）和DeepSeek-V4-Pro（预览版）模型训练好的DSpark检查点。此外，我们开源了DeepSpec，一个算法驱动的训练仓库，包括Eagle3、DFlash和DSpark。这些工件旨在支持关于高效LLM服务的进一步研究。

2. 背景

2.1. 推测解码

自回归语言模型每次前向传播生成一个词元，使得推理延迟与输出长度成正比。推测解码（Chen et al., 2023; Ge et al., 2022; Leviathan et al., 2023）使用轻量级草稿模型 M_d 加速目标模型 M_t 的推理。在每个解码周期，草稿模型提出 γ 个候选词元 $x_1, \dots, x_\gamma$ 。目标模型在单次前向传播中验证所有候选词元，接受与其自身分布一致的最长前缀。

具体来说，在每个草稿位置 k ，目标模型计算其自身分布 p_{tk} ，并将其与草稿分布 p_{dk} 进行比较。词元 x_k 以概率 $\min(1, p_{tk}(x_k)/p_{dk}(x_k))$ 被接受。验证从左到右进行：在位置 k 的第一次拒绝会丢弃所有后续词元 $x_{k+1}, \dots, x_\gamma$ ，无论其质量如何。

设 τ 表示每个周期接收的词元数量， T_{draft} 和 T_{verify} 分别表示草稿传递和验证传递的墙钟时间。每个生成词元的平均延迟为：

$$L = (T_{\text{draft}} + T_{\text{verify}}) / \tau. \quad (1)$$

因此，改进加速比归结为三个杠杆：降低 T_{draft} （更快地草稿），提高 τ （更好地草稿），或降低有效的 T_{verify} （更智能地验证）。

2.2. 草稿模型架构

草稿模型的设计决定了 T_{draft} 和 τ 如何权衡。现有方法分为两类。

自回归草稿模型。

自回归草稿模型顺序生成草稿词元，每个位置以先前采样的词元为条件（DeepSeek-AI, 2024; Li et al., 2024b, c, 2026b; Zhang et al., 2025）。这种显式依赖提供了强大的建模能力，但草稿成本随块大小线性增长： $T_{\text{draft}} \propto \gamma$ ，这迫使自回归草稿模型使用小的 γ 和浅层架构以保持 T_{draft} 较低。为了补偿短块，基于树的验证（Miao et al., 2024）将候选词元扩展成一棵树并通过树注意力验证多条路径，但大量验证词元降低了整体服务吞吐量。

并行草稿模型。

并行草稿模型在单次前向传播中生成所有 γ 个草稿词元，使得 T_{draft} 几乎与块大小无关（Cai et al., 2024; Chen et al., 2026; Li et al., 2025a; Liu et al., 2026a; Sandler et al., 2026）。这允许显著更大的块（例如， $\gamma=16$ ）而不会成比例地增加延迟。

其中，DFlash（Chen et al., 2026）是一个最先进的并行草稿模型，其草稿模型以从目标模型提取的丰富上下文特征为条件（KV注入）。在预填充期间，来自一组目标层 $\{l_1, \dots, l_m\}$ 的隐藏状态被拼接并投影到草稿隐藏空间：

$$H_{\text{ctx}} = \text{RMSNorm}(W_c[H(l_1); \dots; H(l_m)]), \quad (2)$$

其中 $W_c \in \mathbb{R}^{d \times md}$ 是一个共享投影。这些上下文特征通过沿序列维度与草稿块表示拼接（在键和值上）注入到每个草稿层：

$$\begin{aligned} K_i &= [W_{K_i} H_{\text{ctx}}; W_{K_i} H_d], \\ V_i &= [W_{V_i} H_{\text{ctx}}; W_{V_i} H_d]. \end{aligned} \quad (3)$$

块内的所有位置彼此之间以及与注入的目标上下文之间进行双向注意力。草稿模型共享目标模型的嵌入层和语言模型头（均冻结）。它接收一个锚点词元 l 的嵌入后跟 γ 个掩码词元嵌入作为输入，并在单次前向传播中为所有掩码位置生成 logits。由于无论块大小如何，草稿仅需一次前向传播，DFlash 在相同延迟预算下可以比自回归草稿模型采用更深的架构和更大的块。

3. 架构

DSpark 的概览如图1所示。回顾公式1，推测解码的每词元延迟为 $L = (T_{\text{draft}} + T_{\text{verify}}) / \tau$ 。自回归草稿模型实现了高 τ 但付出了 $T_{\text{draft}} \propto \gamma$ 的代价；并行草稿模型将 T_{draft} 压缩为单次传播但牺牲了 τ ，因为每个位置是独立预测的。同时，固定长度的验证将 T_{verify} 浪费在几乎肯定会被拒绝的低置信度后缀词元上。DSpark 通过两个互补组件解决了这些局限性：

- 半自回归生成（第3.1节）。一个并行主干处理草稿计算的主要部分，使 T_{draft} 几乎与 γ 无关。然后一个轻量级顺序块在草稿词元之间注入依赖关系，以最小的额外延迟提高 τ 。
- 基于置信度调度的验证（第3.2节）。一个置信度头估计每个位置的接收概率，一个硬件感知调度器使用这些估计来剪除低置信度后缀词元，减少不必要的验证计算。

1 在本文中，我们互换使用术语锚点词元和奖励词元，表示目标模型在上一解码轮次生成的最终词元。

图1

DSpark 架构和解码周期。给定提示词元 ABC，目标模型执行一步生成下一个词元 D，作为草稿阶段的锚点。以 D 为输入，DSpark 采用一个重型并行主干和一个轻量级顺序头来生成草稿词元 EFGH 及其对应的置信度分数 $c_1 - c_4$ 。然后硬件感知前缀调度器评估这些分数，保留前缀 EFG 并丢弃低置信度词元 H。最后，目标模型并行验证调度后的前缀。如图所示，E 和 F 被接受，而 G 被拒绝，促使模型生成一个纠正后的词元 G^* 以完成当前轮次。

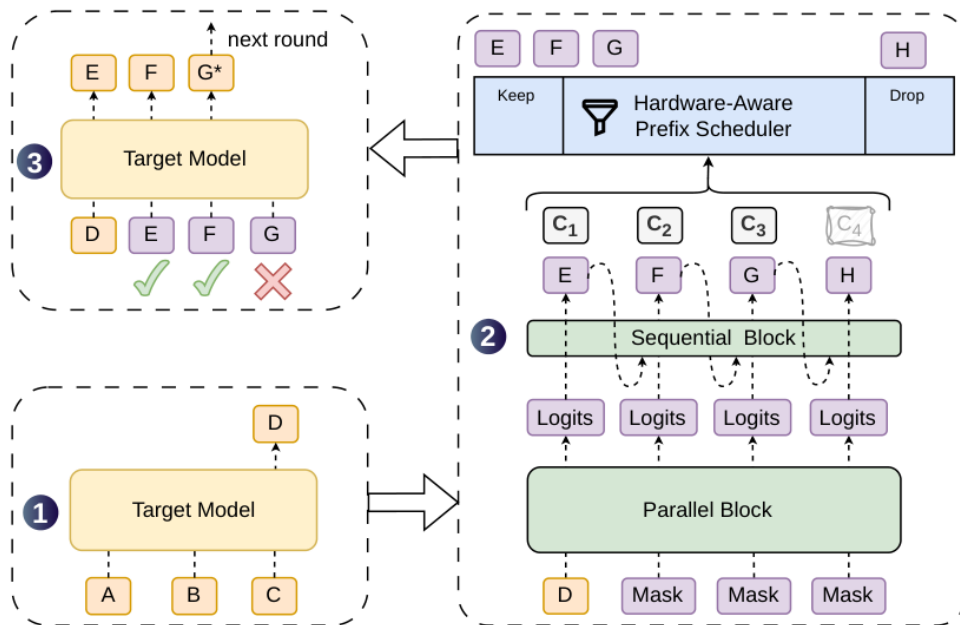


Figure 1 | **The DSspark architecture and decoding cycle.** Given prompt tokens $\boxed{ABC}$, the target model executes one step to generate the next token $\boxed{D}$, which serves as the anchor for the drafting phase. Using $\boxed{D}$ as the input, DSspark employs a heavy parallel backbone and a lightweight sequential head to generate draft tokens $\boxed{EFGH}$ along with their corresponding confidence scores $\boxed{c_1-c_4}$. The Hardware-Aware Prefix Scheduler then evaluates these scores to retain the prefix $\boxed{EFG}$ and drop the low-confidence token $\boxed{H}$. Finally, the target model verifies the scheduled prefix in parallel. As illustrated, $\boxed{E}$ and $\boxed{F}$ are accepted while $\boxed{G}$ is rejected, prompting the model to generate a corrected token $\boxed{G^*}$ to complete the current round.

In combination, the two components let DSspark draft better and verify smarter. We detail each below.

3.1. Semi-Autoregressive Generation

A parallel drafter produces all γ draft logits in one forward pass, so each prediction cannot condition on tokens sampled elsewhere in the block. When the context admits multiple plausible

通过组合，这两个组件使DSspark能够更好地草稿和更智能地验证。我们下面详述每个部分。

3.1. 半自回归生成

一个并行草稿模型在单次前向传播中产生所有 γ 个草稿 logits，因此每个预测不能以块中其他位置采样的词元为条件。当上下文允许多个合理的延续时，例如“of course”和“no problem”，一个并行草稿模型可能产生不连贯的组合，如“of problem”或“no course”，因为每个位置对所有可能的先验词元进行边缘化，而不是以实际采样的那个为条件 (Gut et al., 2018; Huang et al., 2022a)。因此，接收率沿块迅速衰减，浪费了草稿和验证的计算量。因此我们采用一种半自回归结构，将草稿生成分为两个阶段：

并行阶段。

一个并行主干（在我们的实例化中即DFlash (Chen et al., 2026)）对整个块执行单次前向传播，生成隐藏状态 $h_1, \dots, h_\gamma$ 和基础 logits $U_1, \dots, U_\gamma$ 。我们对原始DFlash主干做了一个小修改：不同于输入一个锚点词元加 γ 个掩码词元并仅预测掩码位置，我们将锚点本身作为第一个预测位置，因此 γ 个输入词元（锚点 + $\gamma-1$ 个掩码）产生 γ 个草稿 logits。这减少了草稿计算，同时保持了相似的草稿质量。

顺序阶段。

顺序阶段用依赖于前缀的转移偏置 $B_k(x_0, x_{<k})$ 补充基础logits，允许每个草稿位置以块内先前采样的词元为条件。顺序阶段不是定义一个全局归一化的能量模型，而是通过自回归分解引入一个因果块分布：

$$P(X| x_0) = \prod_{k=1}^K p_k(x_k| x_0, x_{<k}),$$

$$p_k(v| x_0, x_{<k}) = \exp(U_k(v) + B_k(x_0, x_{<k}, v)) / \sum_{u \in V} \exp(U_k(u) + B_k(x_0, x_{<k}, u)). \quad (4)$$

这里， x_0 表示来自上一验证轮次的锚点词元， U_k 是并行主干在位置 k 产生的基础logit向量， V 是词汇表。在推理时，顺序块根据 $p_k(\cdot | x_0, x_{<k})$ 从左到右采样。由于此采样过程本质上是顺序的，该块必须在计算上轻量（ $T_{\text{sequential}} \ll T_{\text{parallel}}$ ），以便整体草稿延迟仍由并行阶段主导。我们在下面描述顺序块的两种实例化。

- 马尔可夫头。最简单的实例化将 B_k 限制为仅依赖于紧邻的前一词元，将其简化为一个一阶转移 $B(x_{k-1}, x_k)$ 。原则上这是一个完整的 $V \times V$ 矩阵 B ；我们使用低秩分解 $B = W_1 W_2$ 来近似，其中 $W_1 \in \mathbb{R}^{V \times r}$ 且 $W_2 \in \mathbb{R}^{r \times V}$ 。给定前一词元 x_{k-1} ，位置 k 的转移偏置为：

$$B(x_{k-1}, \cdot) = W_1[x_{k-1}] W_2 \in \mathbb{R}^V, \quad (5)$$

其中 W_1 作为嵌入查找表， W_2 作为logit投影。低秩分解（默认 $r=256$ ）使存储和每步计算量都很小，即使对于大词汇表也能使顺序循环保持高效。回到之前的例子：一旦位置1采样了“of”，马尔可夫头在位置2提高“course”并抑制“problem”，从而缓解了跨模式碰撞。

•

RNN头。马尔可夫头在超过一步后是无记忆的——位置 k 无法访问 x_{k-1} 之前的词元。RNN头通过维护一个循环状态 s_k 来缓解这一点，该状态在块内累积完整的前缀历史。在每一步，该模块将当前状态 $s_{k-1} \in \mathbb{R}^r$ 、前一词元嵌入 $W_1[x_{k-1}] \in \mathbb{R}^r$ 和主干隐藏状态 $h_k \in \mathbb{R}^d$ 拼接成一个输入向量 $z_k = [s_{k-1}; W_1[x_{k-1}]; h_k] \in \mathbb{R}^{2r+d}$ ，然后应用单步门控更新：

$$s_k = \sigma(W_g z_k) \odot s_{k-1} + (1 - \sigma(W_g z_k)) \odot \tanh(W_c z_k),$$

$$B_k(x_{<k}, \cdot) = W_o \tanh(W_o z_k), \quad (6)$$

其中 $W_g, W_c, W_o \in \mathbb{R}^{(2r+d) \times r}$ 由单个线性投影联合参数化，该投影被分割为门、候选和输出组件。状态 s_0 初始化为零。

3.2. 基于置信度调度的验证

半自回归架构使DSpark能够高效地生成大的草稿块。然而，产生更多的草稿词元并不会自动转化为更高的端到端加速比。不加区分地验证整个草稿块实际上可能降低整体系统吞吐量，特别是在高并发场景下（Huang et al., 2026; Liu et al., 2024c）。

这一性能瓶颈源于两个相互作用的因素。首先，在数据方面，草稿接收率在不同领域间固有地变化：像代码这样的结构化文本自然产生高接收率，而开放性聊天则具有显著较低的接收率（Abramovich et al., 2026; Xia et al., 2024）。其次，在系统方面，验证额外词元的实际成本严格取决于引擎负载。在系统轻负载下，额外的验证即使被拒绝也产生最小的惩罚。然而，在高并发部署中，每一次不必要的验证都占用了本可以服务于其他活跃请求的目标模型批处理容量（Liu et al., 2024b; Wu et al., 2025）。

因此，充分释放大草稿块的潜力需要一个统一的机制，将目标模型计算仅路由到具有正预期回报的词元。DSpark通过将预测前缀存活概率的置信度头（第3.2.1节）与一个基于当前系统负载动态确定最优验证长度的硬件感知前缀调度器（第3.2.2节）相结合来实现这一点。

3.2.1. 置信度头

受Huang et al. (2024); Wang et al. (2026)的启发，置信度头为每个草稿位置 k 输出一个标量估计 $c_k \in (0, 1)$ 。

关键的是， c_k 建模了在块中所有前一词元已被接受的条件下，位置 k 处的草稿词元将通过目标验证的条件概率。该架构包含一个轻量级线性投影后接一个sigmoid函数：

$$c_k = \sigma(w \perp [h_k; W_1[x_{k-1}]]), \quad (7)$$

其中 h_k 是主干的隐藏状态， $W_1[x_{k-1}]$ 是来自前一个草稿词元的马尔可夫嵌入。我们使用每步分析接收率 c_k 来监督 c_k 。该速率由草稿分布 p_{dk} 与目标分布 p_{tk} 之间的总变差距离确定：

$$c_k = 1 - (1/2)^{\|p_k - p_{tk}\|_1} \quad (8)$$

事后校准。

不同于基于阈值的验证启发式方法（Huang et al., 2024; Li et al., 2024b; Zhang et al., 2026b），它们仅需要置信度分数正确排序草稿词元的质量，我们的硬件感知调度方法（详见第3.2.2节）精确地需要累积接收概率的绝对值来计算期望接收长度 τ 。由于神经网络的置信度估计通常过于自信（Guo et al., 2017; Ovadia et al., 2019），直接使用原始置信度分数会扭曲吞吐量估计，导致次优调度。

为了解决这个问题，我们引入了顺序温度缩放（STS）。由于每个 c_i 建模一个条件概率，链式法则规定草稿前缀被接受的联合概率分解为累积乘积 $\prod_{i \leq k} c_i$ 。

使用一个留出验证集，STS从左到右依次校准这个联合概率。具体来说，在每个位置 $k \in \{1, \dots, \gamma\}$ ，我们执行一个简单的一维网格搜索，找到最小化累积乘积的期望校准误差（ECE）（Naeini et al., 2015）的最优温度标量，同时保持所有先前位置已校准的分数不变。关键的是，温度缩放是一个保序变换：它校正预测概率以匹配经验接收率，而不破坏置信度头学习到的相对草稿词元排序。

3.2.2. 硬件感知前缀调度器

算法1 硬件感知前缀调度器

Algorithm 1 Hardware-Aware Prefix Scheduler

Require: Active requests $r \in \{1, \dots, R\}$; confidence sequence $c_{r,1}, \dots, c_{r,\gamma}$ per request; profiled step curve $\text{SPS}(B)$

Ensure: Selected per-request prefix lengths $\ell_1^*, \dots, \ell_R^*$

```
1: for  $r = 1$  to  $R$  do
2:   Compute prefix survival probabilities:  $a_{r,j} \leftarrow \prod_{i \leq j} c_{r,i}$  for  $j = 1, \dots, \gamma$ 
3: end for
4: Construct candidate space  $\mathcal{E} \leftarrow \{(r, j) \mid a_{r,j} > 0\}$  and sort descending by  $a_{r,j}$ 
5: Initialize states:  $\ell_r \leftarrow 0$  for all  $r$ ; Batch size  $B \leftarrow R$ ; Expected accepts  $\tau^* \leftarrow R$ 
6: Initialize tracking:  $\Theta_{\text{best}} \leftarrow R \cdot \text{SPS}(R)$ ; Selected lengths  $\ell_r^* \leftarrow 0$  for all  $r$ 
7: for each  $(r, j) \in \mathcal{E}$  in sorted order do
8:    $\ell_r \leftarrow j$ ;  $B \leftarrow B + 1$ ;  $\tau^* \leftarrow \tau^* + a_{r,j}$ 
9:   Current throughput  $\Theta \leftarrow \tau^* \cdot \text{SPS}(B)$ 
10:  if  $\Theta > \Theta_{\text{best}}$  then
11:     $\Theta_{\text{best}} \leftarrow \Theta$ ; Update selected lengths  $\ell_r^* \leftarrow \ell_r$ 
12:  else
13:    break
14:  end if
15: end for
16: return  $(\ell_1^*, \dots, \ell_R^*)$  achieving  $\Theta_{\text{best}}$ 
```

Prior methods (Huang et al., 2024; Li et al., 2024b) typically apply a static threshold to confidence scores to determine verification length. While effective under isolated, single-request assumptions, static thresholds can be suboptimal in high-concurrency production systems, where the utility of verifying a draft token depends heavily on the current system load.

To address this, we formulate verification length selection as a global throughput maximization problem (Algorithm 1). Consider a batch of R active requests. For request r , let $c_{r,1}, \dots, c_{r,\gamma}$ be the per-position confidence estimates, and let $\ell_r \in \{0, \dots, \gamma\}$ denote the scheduled verification length. Because speculative decoding dynamically accepts draft tokens only as a continuous prefix, the survival probability of a token at position j is the cumulative product $a_{r,j} = \prod_{i \leq j} c_{r,i}$.

In a single verification step, the total batch size (measured in tokens) sent to the target model is $B = \sum_{r=1}^R (1 + \ell_r)$, and the expected number of successfully accepted tokens is $\tau = \sum_{r=1}^R (1 + \sum_{j=1}^{\ell_r} a_{r,j})$. Let $\text{SPS}(B)$ denote the engine throughput, measured in steps per second, for a given forward-pass batch size B . Crucially, this capacity curve is profiled once during engine initialization and stored as a lightweight cost table. Our scheduler then aims to maximize the expected system-wide token throughput $\Theta = \tau \cdot \text{SPS}(B)$ by dynamically selecting verification lengths $\ell_1, \dots, \ell_R$.

Although finding the global maximum of Θ appears to be a combinatorial search, the objective structure allows for an efficient greedy solution. Because $a_{r,j}$ is monotonically non-increasing with respect to j (i.e., $a_{r,j} \leq a_{r,j-1}$), the marginal gain in expected accepted tokens for extending request r 's verification length from $j-1$ to j is exactly $a_{r,j}$. This monotonicity ensures that sorting candidate tokens globally by $a_{r,j}$ naturally respects intra-block prefix dependencies. Consequently, if the total verification batch size B were fixed, the optimal allocation $\{\ell_r\}$ would be determined by greedily selecting the draft tokens with the highest survival probabilities from the global pool of all $\{a_{r,j}\}$.

Building on this insight, the optimization can be evaluated along this greedy admission path.

要求: 活跃请求 $r \in \{1, \dots, R\}$; 每个请求的置信度序列 $c_{r,1}, \dots, c_{r,\gamma}$; 预分析的步数曲线 $\text{SPS}(B)$
确保: 选定的每请求前缀长度 $\ell_1^*, \dots, \ell_R^*$

```

1: for  $r = 1$  to  $R$  do
2:   计算前缀存活概率:  $ar, j \leftarrow \prod_{i \leq j} cr, i$  对于  $j = 1, \dots, \gamma$ 
3: end for
4: 构建候选空间  $E \leftarrow \{(r, j) \mid ar, j > 0\}$  并按  $ar, j$  降序排序
5: 初始化状态:  $lr \leftarrow 0$  对于所有  $r$ ; 批大小  $B \leftarrow R$ ; 期望接收数  $\tau^* \leftarrow R$ 
6: 初始化追踪:  $\Theta_{best} \leftarrow R \cdot \text{SPS}(R)$ ; 选定长度  $l^*r \leftarrow 0$  对于所有  $r$ 
7: for each  $(r, j) \in E$  in sorted order do
8:    $lr \leftarrow j$ ;  $B \leftarrow B + 1$ ;  $\tau^* \leftarrow \tau^* + ar, j$ 
9:   当前吞吐量  $\Theta \leftarrow \tau^* \cdot \text{SPS}(B)$ 
10:   if  $\Theta > \Theta_{best}$  then
11:      $\Theta_{best} \leftarrow \Theta$ ; 更新选定长度  $l^*r \leftarrow lr$ 
12:   else
13:     break
14:   end if
15: end for
16: return  $(l^*1, \dots, l^*R)$  达到  $\Theta_{best}$ 

```

先前的方法 (Huang et al., 2024; Li et al., 2024b) 通常对置信度分数应用静态阈值来确定验证长度。虽然在孤立的单请求假设下有效,但在高并发生产系统中静态阈值可能是次优的,其中验证一个草稿词元的效用很大程度上取决于当前系统负载。

为了解决这个问题,我们将验证长度选择形式化为一个全局吞吐量最大化问题(算法1)。考虑一批 R 个活跃请求。对于请求 r ,设 $cr, 1, \dots, cr, \gamma$ 为逐位置置信度估计,并设 $lr \in \{0, \dots, \gamma\}$ 表示调度的验证长度。由于推测解码仅作为连续前缀动态接收草稿词元,位置 j 处词元的存活概率是累积乘积 $ar, j = \prod_{i \leq j} cr, i$ 。

在单次验证步骤中,发送给目标模型的总批大小(以词元计)为 $B = \sum_{r=1}^R (1 + lr)$,成功接收的期望词元数为 $\tau = \sum_{r=1}^R (1 + \sum_{j=1}^{lr} ar, j)$ 。设 $\text{SPS}(B)$ 表示对于给定前向传播批大小 B 的引擎吞吐量,以每秒步数衡量。关键的是,该容量曲线在引擎初始化期间被分析一次,并存储为一个轻量级成本表。然后我们的调度器旨在通过动态选择验证长度 $l_1, \dots, l_R$ 来最大化期望的系统范围词元吞吐量 $\Theta = \tau \cdot \text{SPS}(B)$ 。

尽管找到 Θ 的全局最大值看似是一个组合搜索,但目标结构允许一个高效的贪心解法。由于 ar, j 相对于 j 是单调非增的(即, $ar, j \geq ar, j-1$),将请求 r 的验证长度从 $j-1$ 扩展到 j 的期望接收词元边际增益恰好是 ar, j 。这种单调性确保全局按 ar, j 排序候选词元自然尊重块内前缀依赖关系。因此,如果总验证批大小 B 是固定的,最优分配 $\{l_r\}$ 将由从所有 $\{ar, j\}$ 的全局池中贪心地选择具有最高存活概率的草稿词元来确定。

基于这一洞察,优化可以沿着这条贪心准入路径进行评估。我们首先将所有有效的前缀扩展按存活概率降序全局排序。为了动态确定最优的目标批大小 B ,我们逐步从这个排序池中准入词元,通过从预分析的成本表中进行 $O(1)$ 查找来更新期望吞吐量 Θ 。

无损推测解码严格要求非预期属性:准入决策不得依赖于未来的候选词元(Chen et al., 2023; Leviathan et al., 2023)。由于我们的置信度头依赖于先前采样词元的马尔可夫特征,计算下一个存活概率 $ar, k+1$ 明确需要实例化的候选词元 xr, k 。因此,回顾性全局搜索会无意中将 xr, k 泄露到第 k 步的准入决策中,引入选择偏置(我们在附录A中提供了一个具体反例来证明这一理论违反)。

为了强制严格因果性,调度器(算法1)采用了一种早停机制。通过在吞吐量下降时($\Theta \leq \Theta_{best}$)立即中断贪心搜索,截断决策仅依赖于处理到该确切步骤的前缀。这将准入事件与未来词元隔离开,确保了精确的目标分布恢复。注意,这种逐步早停当且仅当目标 Θ 是单峰时才产生全局最大吞吐量,这隐含地假设了一个平滑衰减的硬件容量曲线。我们在第5.2节中讨论针对真实世界非平滑SPS特性和异步系统流水线所需的工程适配。

3.3. 训练

在训练期间,我们从每个目标序列中随机采样多个锚点位置,形成 γ 词元块作为训练数据。目标模型在整个训练过程中被冻结;草稿模型共享其嵌入层和语言模型头并保持它们冻结,仅更新主干草稿器、顺序块和置

信度头。

训练目标由三项组成：一个交叉熵损失 L_{ce} 、一个分布匹配损失 L_{tv} 和一个置信度损失 L_{conf} 。所有三项均按位置加权，权重为 $w_k = \exp(-(k-1)/\gamma)$ (Chen et al., 2026)，这更强调在基于前缀的验证下对期望接收长度贡献更大的早期块位置。交叉熵损失 L_{ce} 训练草稿器预测正确的下一个词元：

$$L_{ce} = -\sum_{k=1}^{\infty} w_k \log p_{dk}(x^*k), \quad (9)$$

其中 x^*k 是真实词元， p_{dk} 是草稿分布。分布匹配损失 L_{tv} 惩罚草稿分布与目标分布之间的总变差距离：

$$L_{tv} = \sum_{k=1}^{\infty} w_k \|p_{dk} - p_{tk}\|_1. \quad (10)$$

由于总变差距离是接收率的直接代理：每步接收概率等于 $1 - (1/2) \|p_d - p_t\|_1$ (Leviathan et al., 2023)，最小化 L_{tv} 直接最大化期望接收率。置信度损失 L_{conf} 是一个二值交叉熵，训练置信度头预测来自公式8的软接收标签 c^*k ：

$$L_{conf} = -\sum_{k=1}^{\infty} w_k [c^*k \log c_k + (1 - c^*k) \log(1 - c_k)]. \quad (11)$$

总体目标是三项的加权组合（默认权重 $\alpha_{ce} = 0.1$, $\alpha_{tv} = 0.9$, $\alpha_{conf} = 1.0$ ）：

$$L = \alpha_{ce} L_{ce} + \alpha_{tv} L_{tv} + \alpha_{conf} L_{conf} \quad (12)$$

4. 实验

在本节中，我们使用离线基准测试验证DSpark的草稿质量，并在第5节中报告置信度调度器在在线生产流量下的有效性。实验设置在第4.1节中描述，主要结果在第4.2节中，以及额外分析见第4.3节。

4.1. 实验设置

目标模型和草稿模型。

我们在跨越不同规模和模型族的四个目标模型上评估 DSpark: Qwen3-{4B, 8B, 14B} (Yang et al., 2025) 和 Gemma4-12B (Google DeepMind, 2026)。对于草稿模型，我们将 DSpark 与两个代表性起草器进行比较：DFlash (Chen et al., 2026)，一个最先进的并行起草器，以及 Eagle3 (Li et al., 2026b)，一个基于训练时测试 (TTT) 的自回归起草器。为了公平比较，我们在相同的训练框架和相同的数据上重新训练所有起草器。我们将 Eagle3 的 TTT 视野 (7) 与 DFlash 和 DSpark 使用的块大小 (7) 对齐，并且我们对所有起草器使用相同的目标模型特征层。对于草稿模型层数，我们为 Eagle3 设置 1 层，为 DSpark 和 DFlash 设置 5 层 (Chen et al., 2026)。除非另有说明，DSpark 指马尔可夫头变体；我们在第 4.3.2 节研究 RNN 头变体。

训练数据。

我们使用 Open-PerfectBlend 2，这是 PerfectBlend (Xu et al., 2024) 的一个开源版本，包含 130 万个样本。它是一个通用指令数据集，包含聊天 (17.6%)、数学 (39.4%)、代码 (38.9%) 和指令跟随数据 (4.1%)。我们只使用 Open-PerfectBlend 的提示；回复由每个目标模型使用推荐的采样参数重新生成。每个起草器训练 10 个 epoch 以确保完全收敛。对于数据生成和评估，我们采用非思考模式。

评估协议。

我们在三个域上评估不同算法的性能：

1. 数学推理，包括 GSM8K (Cobbe et al., 2021)、MATH500 (Lightman et al., 2024) 和 AIME25 (Zhang and Math-AI, 2025)。
2. 代码生成，包括 MBPP (Austin et al., 2021b)、HumanEval (Chen et al., 2021) 和 Live-CodeBench (Jain et al., 2025)。
3. 日常聊天，包括 MT-Bench (Zheng et al., 2023)、Alpaca (Taori et al., 2023) 和 Arena-Hard (Li et al., 2024a, 2025b)。

对于所有基准测试，我们使用标准推测解码 (Chen et al., 2023; Leviathan et al.,

2023), 采样温度设置为
(τ)3。对于所有起草器, 我们使用基于链的起草。

1.0。我们报告每轮解码的接受长度

2<https://huggingface.co/datasets/mlabonne/open-perfectblend>

3为清晰起见, 除非另有说明, 所有报告的接受长度和接受率指标均包含目标模型生成的奖励 token。

=== PAGE BREAK ===

表 1 | 主要推测解码结果。我们报告不同目标模型和域上每轮解码的接受长度
(τ) (越高越好)。粗体表示最佳结果。

Qwen3-4B	Eagle3	5.14	4.62	3.92	3.69	4.16	3.77	2.39	2.26	2.55
	DFlash	5.40	4.85	4.15	4.40	4.74	4.18	3.07	2.96	2.83
	DSpark	6.11	5.70	4.89	5.13	5.38	4.86	3.64	3.54	3.29
Qwen3-8B	Eagle3	5.30	4.77	3.91	3.96	4.33	4.17	2.66	2.54	2.54
	DFlash	5.33	4.91	4.07	4.36	4.64	4.39	3.11	2.98	2.81
	DSpark	6.17	5.78	5.01	5.16	5.52	5.17	3.72	3.58	3.21
Qwen3-14B	Eagle3	5.24	4.60	3.71	3.81	4.14	4.01	2.62	2.47	2.48
	DFlash	5.41	4.84	3.98	4.44	4.59	4.33	3.10	2.94	2.72
	DSpark	6.21	5.74	4.94	5.26	5.43	5.02	3.70	3.58	3.13
Gemma4-12B	Eagle3	5.87	5.46	4.83	4.72	5.37	4.16	3.19	3.06	2.72
	DFlash	5.45	5.04	4.22	4.39	4.95	3.70	2.98	2.84	2.59
	DSpark	6.05	5.78	5.12	5.11	5.64	4.51	3.49	3.35	2.92

4.2. Experimental Results

To isolate the raw draft quality from system-level scheduling policies, our offline evaluation disables the confidence scheduler, forcing all drafters to propose a fixed block of tokens. The main results, measured by the average accepted length (τ) per round, are reported in Table 1.

DSpark consistently outperforms both the autoregressive baseline (Eagle3) and the parallel baseline (DFlash) across all evaluated target models and benchmark domains. Specifically, across the Qwen3-4B, 8B, and 14B models, DSpark improves the macro-average accepted length over Eagle3 by 30.9%, 26.7%, and 30.0%, respectively. Similarly, compared to DFlash, DSpark yields relative improvements of 16.3%, 18.4%, and 18.3% across the three scales. Crucially, this advantage generalizes across model families, as demonstrated by the consistent performance gains on the Gemma4-12B target.

Beyond the average improvements, Table 1 reveals a strong domain effect: the accepted length is naturally higher on structured tasks (e.g., 5.57 on math and 5.12 on code for Qwen3-4B) than on open-ended chat (3.49). This inherent variance in data predictability means a static verification length often wastes compute on trailing tokens that are highly likely to be rejected. This directly motivates our confidence-scheduled verification, which dynamically prunes the draft block based on expected acceptance.

4.3. Experimental Analysis

4.3.1. Why Can Parallel Generation Outperform Autoregression?

Table 1 presents a counter-intuitive observation: the parallel drafter (DFlash) and the semi-autoregressive drafter (DSpark) often yield longer accepted lengths than the fully autoregressive drafter (Eagle3). This finding contrasts with the standard expectation that step-by-step autoregression produces higher-quality sequences than parallel models (Israel et al., 2026; Ren et al., 2020; Zheng et al., 2025).

To analyze this behavior, we examine performance beyond the macro-level accepted length. Using the Qwen3-4B target model and the benchmark sets described in Section 4.1, we introduce *position-wise conditional acceptance* tracked during actual speculative decoding rollouts. Specifically, for a given draft position k , the evaluation denominator counts only the instances where

目标模型	起草器	数学		代码		聊天				
---	---	---	---	---	---	---	---	---	---	---
		GSM8K	MATH	AIME25	MBPP	HumanEval	LCB	MT-Bench	Alpaca	Arena-Hard
Qwen3-4B	Eagle3	5.14	4.62	3.92	3.69	4.16	3.77	2.39	2.26	2.55
	DFlash	5.40	4.85	4.15	4.40	4.74	4.18	3.07	2.96	2.83
	DSpark	6.11	5.70	4.89	5.13	5.38	4.86	3.64	3.54	3.29
Qwen3-8B	Eagle3	5.30	4.77	3.91	3.96	4.33	4.17	2.66	2.54	2.54
	DFlash	5.33	4.91	4.07	4.36	4.64	4.39	3.11	2.98	2.81
	DSpark	6.17	5.78	5.01	5.16	5.52	5.17	3.72	3.58	3.21

	Qwen3-14B		Eagle3		5.24		4.60		3.71		3.81		4.14		4.01		2.62		2.47		2.48	
		DFlash		5.41		4.84		3.98		4.44		4.59		4.33		3.10		2.94		2.72		
		DSpark		6.21		5.74		4.94		5.26		5.43		5.02		3.70		3.58		3.13		
	Gemma4-12B		Eagle3		5.87		5.46		4.83		4.72		5.37		4.16		3.19		3.06		2.72	
		DFlash		5.45		5.04		4.22		4.39		4.95		3.70		2.98		2.84		2.59		
		DSpark		6.05		5.78		5.12		5.11		5.64		4.51		3.49		3.35		2.92		

4. 2. 实验结果

为了将原始草稿质量与系统级调度策略分离开来，我们的离线评估禁用了置信度调度器，强制所有起草器提出固定数量的 token 块。主要结果以每轮平均接受长度 (τ) 衡量，报告在表 1 中。DSpark 在所有评估的目标模型和基准域上始终优于自回归基线 (Eagle3) 和并行基线 (DFlash)。具体来说，在 Qwen3-4B、8B 和 14B 模型上，DSpark 相较于 Eagle3 的宏平均接受长度分别提高了 30.9%、26.7% 和 30.0%。类似地，与 DFlash 相比，DSpark 在三个规模上分别产生了 16.3%、18.4% 和 18.3% 的相对提升。至关重要的是，这一优势跨模型族泛化，正如在 Gemma4-12B 目标上的一致性能增益所证明的那样。

除了平均改进之外，表 1 还揭示了一个强烈的域效应：结构化任务上的接受长度自然更高（例如，Qwen3-4B 上数学为 5.57，代码为 5.12），而开放式聊天则较低（3.49）。这种数据可预测性的固有方差意味着，静态的验证长度通常会将计算浪费在极有可能被拒绝的尾部 token 上。这直接激励了我们的置信度调度验证，它基于期望的接受度动态修剪草稿块。

4. 3. 实验分析

4. 3. 1. 为什么并行生成能优于自回归？

表 1 呈现了一个反直觉的观察：并行起草器 (DFlash) 和半自回归起草器 (DSpark) 通常比完全自回归起草器 (Eagle3) 产生更长的接受长度。这一发现与标准的预期——即逐步自回归比并行模型产生更高质量的序列 (Israel et al., 2026; Ren et al., 2020; Zheng et al., 2025)——形成对比。

为了分析这种行为，我们考察了宏观层面接受长度之外的性能。使用 Qwen3-4B 目标模型和第 4.1 节中描述的基准集，我们引入了在实际推测解码展开期间追踪的逐位置条件接受度。具体来说，对于给定的草稿位置 k，评估分母仅计算目标模型成功验证并接受所有从 1 到 k-1 的前序草稿 token 的实例。该指标然后计算在这些有效实例中位置 k 的 token 也被接受的比例。这种方法确保位置 k 的评估不会因较早的前缀错误而受到惩罚，从而揭示每个特定步骤的潜在预测质量。图 2 详细展示了这些测量结果，清楚地展示了不同架构之间的行为差异。

=== PAGE BREAK ===

	1		2		3		4		5		6		7	
	---		---		---		---		---		---		---	
	0.75		0.80		0.85		0.90		0.95					

条件接受度
数学

	1		2		3		4		5		6		7	
	---		---		---		---		---		---		---	
	0.70		0.75		0.80		0.85		0.90		0.95			

代码

	1		2		3		4		5		6		7	
	---		---		---		---		---		---		---	
	0.50		0.55		0.60		0.65		0.70		0.75		0.80	

聊天
草稿位置

DFlash
Eagle3
DSpark

图 2 | 逐位置条件接受度。我们报告每个草稿位置的经验条件接受率，在使用 Qwen3-4B 目标模型的情况下，对每个域内的基准测试取平均值。与标准的前缀存活率不同，该指标通过移除先前拒绝的惩罚来隔离位置 k 的基线预测质量。注意，自回归起草器（Eagle3）保持稳定或呈上升趋势，而并行起草器（DFlash）则遭受后缀衰减。

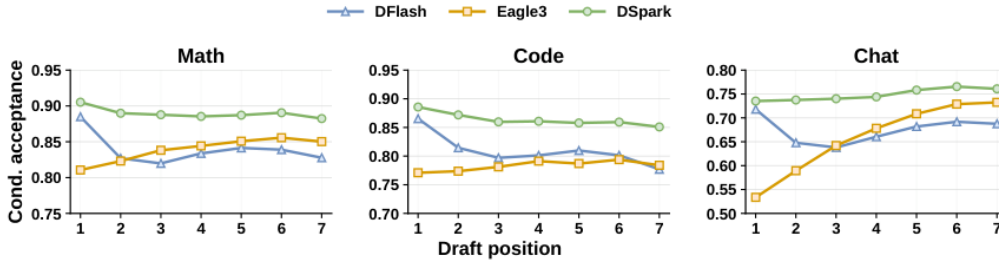


Figure 2 | **Position-wise conditional acceptance.** We report the empirical conditional acceptance rate for each draft position, averaged across benchmarks within each domain using the Qwen3-4B target model. Unlike standard prefix survival, this metric isolates the baseline predictive quality at position k by removing the penalty of previous rejections. Notice that the autoregressive drafter (Eagle3) remains stable or trends upward, while the parallel drafter (DFlash) suffers suffix decay.

the target model successfully verifies and accepts all preceding draft tokens from 1 to $k - 1$. The metric then calculates the proportion of these valid instances where the token at position k is also accepted. This approach ensures that the evaluation of position k is not penalized by earlier prefix errors, revealing the underlying predictive quality at each specific step. Figure 2 details these measurements, demonstrating clear behavioral differences across the architectures.

The Capacity Advantage at Position 1. At the first draft position, both architectures predict the next token based solely on the target context. The performance divergence here stems strictly from architectural capacity: autoregressive models like Eagle3 are constrained to shallow networks due to their $O(\gamma)$ latency, whereas $O(1)$ parallel drafters can afford much deeper networks. This structural gap yields a substantial accuracy margin at position 1, with DFlash starting noticeably higher than Eagle3 (e.g., 0.88 vs. 0.81 on Math, and 0.72 vs. 0.53 on Chat). Because speculative decoding operates as a strict prefix-matching survival process, the first token carries the highest leverage—a rejection here immediately invalidates the entire block. Consequently, this initial capacity advantage disproportionately boosts the final accepted length, explaining why parallel drafters ultimately outperform autoregressive ones globally despite rapid acceptance decay at later positions.

The Limitation of Independence at Later Positions. Examining the tail of the curves (positions 2 through 7) exposes the inherent limitation of independent parallel generation. As earlier tokens lock in a specific semantic path, subsequent tokens naturally become more predictable. Autoregressive models like Eagle3 effectively leverage this conditional certainty, maintaining or even increasing conditional acceptance deeper into the block (e.g., from 0.53 to 0.74 on Chat). In contrast, DFlash suffers from rapid acceptance decay, dropping from 0.87 to 0.78 on Code and 0.72 to 0.63 on Chat. Because each parallel position marginalizes over all possible prior tokens rather than conditioning on an exact sampled prefix, the model frequently proposes inconsistent suffix combinations—a mode known as multi-modal collision (Gu et al., 2018; Stern et al., 2018).

Mitigating Suffix Decay with Semi-Autoregression. The preceding analysis highlights a clear architectural objective: combining the high capacity of a parallel backbone for the initial token with the dependency modeling of an autoregressive model for subsequent tokens. This directly

位置 1 的能力优势。

在第一个草稿位置，两种架构都仅基于目标上下文预测下一个

token。这里的性能差异严格源于架构能力：自回归模型如 Eagle3 因其 $O(\gamma)$ 延迟而受限与浅层网络，而 $O(1)$ 并行起草器可以负担得起更深得多的网络。这种结构性差距在位置

产生了显著的准确率裕度，DFlash 的起点明显高于 Eagle3（例如，数学上 0.88 对比 0.81，聊天上 0.72 对比 0.53）。由于推测解码作为一个严格的前缀匹配存活过程运作，第一个 token 承载着最高的杠杆作用——在此处的拒绝立即使整个块无效。因此，这种初始能力优势不成比例地提升了最终接受长度，解释了为什么并行起草器最终在全局上优于自回归起草器，尽管在后续位置接受度迅速衰减。

后续位置独立性的局限性。

检查曲线尾部（位置 2 到 7）揭示了独立并行生成的固有限制。随着早期 token 锁定特定的语义路径，后续 token 自然变得更加可预测。自回归模型如 Eagle3 有效地利用了这种条件确定性，在块的更深位置保持甚至增加条件接受度（例如，在聊天上从 0.53 增加到 0.74）。相比之下，DFlash 遭受快速接受衰减，在代码上从 0.87 下降到 0.78，在聊天上从 0.72 下降到 0.63。因为每个并行位置是对所有可能的前序 token 进行边缘化，而不是条件依赖于一个精确采样的前缀，该模型经常提出不一致的后缀组合——这种模式被称为多模态碰撞（Gu et al., 2018; Stern et al., 2018）。

用半自回归缓解后缀衰减。

前述分析突出了明确的架构目标：将并行骨干网络对初始 token 的高能力与自回归模型对后续 token 的依赖建模结合起来。这直接激发了 DSpark 的半自回归设计。如图 2 所示，DSpark 继承了深层并行起草器的高初始接受度（例如，在数学上从 0.93 开始）。同时，其轻量级顺序头缓解了并行生成典型的快速接受衰减。通过解决这种权衡，DSpark 在整个草稿块中保持高且稳定的条件接受率。

=== PAGE BREAK ===

	GSM8K	MATH500	AIME25	MBPP	HumanEval	LiveCodeBench	MT-Bench	Alpaca	Arena-Hard v2
接受长度	2	3	4	5	6				

接受长度

方法

DSpark

DFlash

深度

1L

2L

5L

图 3 | 起草器深度的影响。在提案长度固定的情况下，DSpark 的性能随着起草器层数的增加而提高。值得注意的是，浅层 2 层 DSpark 优于更深的 5 层 DFlash 基线，突显了顺序建模的参数效率。

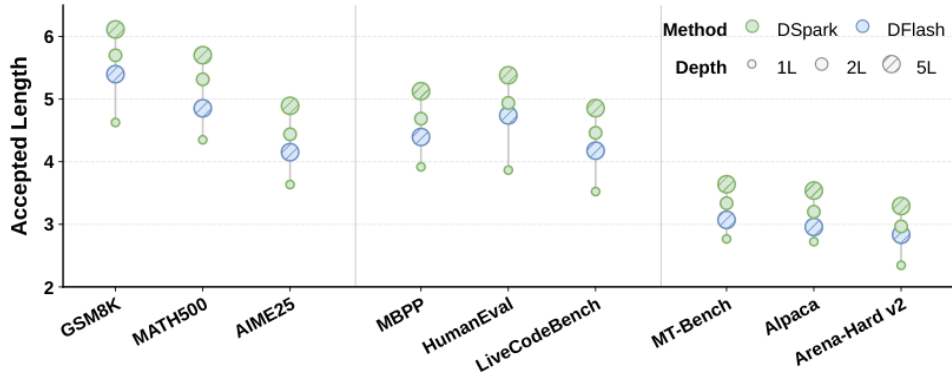


Figure 3 | **Effect of drafter depth.** With proposal length fixed, DSpark’s performance improves as drafter layers are added. Notably, a shallow 2-layer DSpark outperforms a deeper 5-layer DFlash baseline, highlighting the parameter efficiency of sequential modeling.

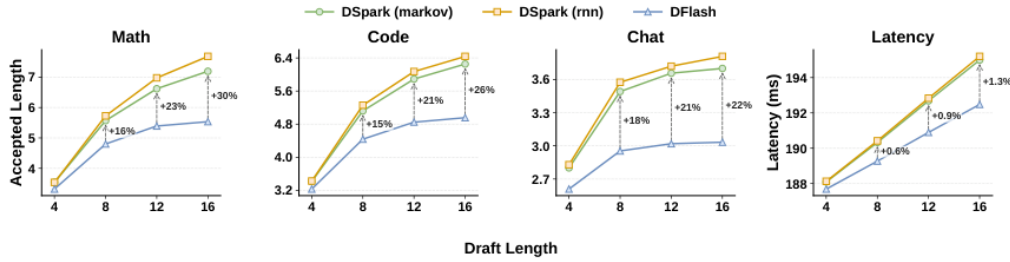


Figure 4 | **Effect of proposal length and latency overhead.** DSpark consistently outperforms DFlash across various block sizes (left three panels). The rightmost panel demonstrates that the sequential head introduces minimal latency overhead during serving.

motivates DSpark’s semi-autoregressive design. As shown in Figure 2, DSpark inherits the high initial acceptance of the deep parallel drafter (e.g., starting at 0.93 on Math). Simultaneously, its lightweight sequential head mitigates the rapid acceptance decay typical of parallel generation. By resolving this trade-off, DSpark maintains a high and stable conditional acceptance rate throughout the entire draft block.

4.3.2. A Little Autoregression Goes a Long Way

Building on the insights from Section 4.3.1, we explore the architectural design space of DSpark along two dimensions: drafter depth (number of transformer layers) and proposal length (block size γ). Unless otherwise stated, all experiments in this section use Qwen3-4B as the target model and follow the evaluation protocol detailed in Section 4.1.

Drafter Depth. Increasing the number of transformer layers naturally expands a draft model’s predictive capacity. To isolate this effect, we fix the block size to 7 and vary the number of DSpark layers from 1 to 5, comparing it against a 5-layer DFlash baseline. Figure 3 aggregates the accepted lengths across the math, code, and chat domains. As expected, DSpark’s performance improves monotonically with depth, with the steepest marginal gain occurring from one to two layers. Notably, a 2-layer DSpark outperforms the 5-layer DFlash baseline across all domains.

4	8	12	16
---	---	---	---
4	5	6	7

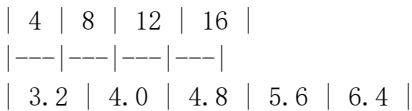
接受长度

数学

+16%

+23%

+30%

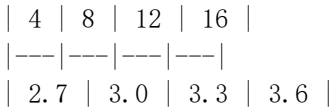


代码

+15%

+21%

+26%

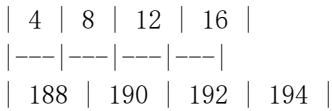


聊天

+18%

+21%

+22%



延迟 (ms)

延迟

+0.6%

+0.9%

+1.3%

草稿长度

DSpark (markov)

DSpark (rnn)

DFlash

图 4 | 提案长度和延迟开销的影响。DSpark 在各种块大小上始终优于 DFlash（左侧三个面板）。最右侧面板展示了顺序头在服务期间引入了最小的延迟开销。

This demonstrates that injecting local auto-regression via a lightweight sequential head offers a highly favorable accuracy-parameter trade-off, achieving better sequence coherence than simply stacking deeper parallel layers.

Proposal Length. Next, we fix the drafter depth to 5 layers and scale the draft length (proposal length γ plus one anchor token) across $\{4, 8, 12, 16\}$ to evaluate performance on longer draft blocks. For DSpark, we evaluate both the default Markov head and the RNN head. The first three panels of Figure 4 show that DSpark consistently outperforms DFlash at every proposal length. More importantly, the performance gap steadily widens as γ increases. Because pure parallel generation (DFlash) suffers from rapid acceptance decay (Figure 2), its marginal utility diminishes for long blocks. DSpark mitigates this decay, causing its relative gain over DFlash to grow. For instance, at $\gamma = 7$, DSpark improves the accepted length by 16% on math, 15% on code, and 18% on chat; at $\gamma = 15$, these gains expand to 30%, 26%, and 22%, respectively. Also, RNN head provides only marginal additional gains over the Markov head, mainly at longer proposal lengths. Given its higher implementation complexity and less favorable deployment properties, we use the Markov head as the default.

Latency Overhead. We quantify the overhead of the sequential generation loop in DSpark. The rightmost panel of Figure 4 reports the per-round engine latency—comprising one target verification pass, the parallel draft block forward, and the serial sampling loop—measured at a batch size of 128. To prevent sequence-length bias, the reported latency represents the arithmetic mean across varying context lengths ($\{512, 1024, 2048, 4096\}$ tokens). Since the target model dominates the verification compute time at this batch size, the sequential block’s latency overhead is negligible. Consequently, scaling the draft length from 4 to 16 adds a marginal 0.2% to 1.3% to the full-round latency over the DFlash baseline, despite delivering up to a 30% improvement in accepted length.

4.3.3. Verify Smarter, Not Longer: The Role of Confidence Head

While DSpark sustains high acceptance over long draft blocks, verifying the entire proposal remains inefficient (Hu et al., 2026; Huang et al., 2024). Due to the inherent domain variance noted in Section 4.2, trailing tokens in open-ended chat still face high rejection risks, making blind verification a waste of target compute. To evaluate whether the confidence head can effectively prune these unpromising suffixes, we conduct an offline threshold sweep using Qwen3-4B. We validate the estimator in isolation here, reserving the hardware-aware prefix scheduler (Section 3.2.2) for live production evaluation in Section 5.

Diagnostic: Static Threshold Sweep. Figure 5 plots the average tokens per step (bars) and the overall acceptance rate (line) across confidence thresholds. As the threshold increases, the acceptance rate steadily rises because the estimator filters out tokens that would ultimately be rejected (hashed bars). This suggests that the confidence head can identify lower-value suffix tokens and this pruning is most pronounced on chat workloads, where higher-entropy token distributions limit the efficiency of fixed-length verification. In the Chat subplot, raising the threshold significantly reduces rejected tokens, increasing the acceptance rate from 45.7% to 95.7%. In contrast, structured tasks (Math and Code) experience milder pruning and retain more draft tokens, with acceptance rates rising from 76.9% to 92.5% and 67.6% to 92.0%, respectively.

4.3.2. 一点自回归就能走得很远

基于第 4.3.1 节的洞见，我们沿着两个维度探索 DSpark 的架构设计空间：起草器深度（Transformer 层数）和提案长度（块大小 γ ）。除非另有说明，本节中的所有实验均使用 Qwen3-4B

作为目标模型，并遵循第 4.1 节详述的评估协议。

起草器深度。

增加 Transformer 层数自然会扩展草稿模型的预测能力。为了隔离这种效应，我们将块大小固定为 7，并将 DSpark 的层数从 1 变化到 5，与 5 层 DFlash 基线进行比较。图 3 汇总了数学、代码和聊天域上的接受长度。正如预期的那样，DSpark 的性能随深度单调提升，最陡峭的边际增益出现在从一层到两层。值得注意的是，2 层 DSpark 在所有域上都优于 5 层 DFlash 基线。这表明，通过轻量级顺序头注入局部自回归提供了非常有利的精度-参数权衡，实现了比简单堆叠更深的并行层更好的序列连贯性。

提案长度。

接下来，我们将起草器深度固定为 5 层，并将草稿长度（提案长度 γ 加上一个锚点 token）在 {4, 8, 12, 16} 范围内缩放，以评估在更长草稿块上的性能。对于 DSpark，我们评估默认的马尔可夫头和 RNN 头两者。图 4 的前三个面板显示，DSpark 在每个提案长度上始终优于 DFlash。更重要的是，随着 γ 的增加，性能差距稳步扩大。由于纯并行生成（DFlash）遭受快速接受衰减（图 2），其边际效用对于长块而言逐渐减小。DSpark 缓解了这种衰减，导致其相对于 DFlash 的相对增益增长。例如，在 $\gamma = 7$ 时，DSpark 在数学上提高了接受长度 16%，在代码上提高 15%，在聊天上提高 18%；在 $\gamma = 15$ 时，这些增益分别扩大到 30%、26% 和 22%。此外，RNN 头相对于马尔可夫头仅提供边际额外增益，主要是在更长的提案长度上。鉴于其更高的实现复杂性和不太有利的部署特性，我们使用马尔可夫头作为默认配置。

延迟开销。

我们量化 DSpark 中顺序生成循环的开销。图 4 的最右侧面板报告了每轮引擎延迟——包括一次目标验证传递、并行草稿块前向传播和串行采样循环——在批大小为 128 的情况下测量。为防止序列长度偏差，报告的延迟代表不同上下文长度（{512, 1024, 2048, 4096} token）的算术平均值。由于在此批大小下目标模型主导验证计算时间，顺序块的延迟开销可以忽略不计。因此，将草稿长度从 4 扩展到 16 仅在全轮延迟上比 DFlash 基线增加 0.2% 到 1.3%，尽管在接受的 token 长度上带来了高达 30% 的改善。

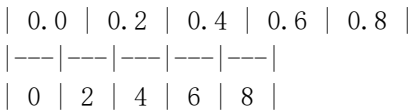
4.3.3. 更智能地验证，而非更长：置信度头的作用

虽然 DSpark 在长草稿块上维持高接受度，但验证整个提案仍然低效（Hu et al., 2026; Huang et al., 2024）。由于第 4.2 节中指出的固有域方差，开放式聊天中的尾部 token 仍然面临高拒绝风险，使得盲目验证浪费目标计算资源。为了评估置信度头是否能有效修剪这些没有前途的后缀，我们使用 Qwen3-4B 进行了离线阈值扫描。我们在此孤立地验证估计器，将硬件感知前缀调度器（第 3.2.2 节）留待第 5 节的在线生成评估。

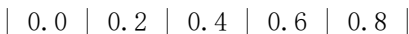
诊断：静态阈值扫描。

图 5 绘制了跨越置信度阈值的每步平均 token 数（柱状图）和整体接受率（折线图）。随着阈值的增加，接受率稳步上升，因为估计器过滤掉了最终会被拒绝的 token（斜线阴影柱状图）。这表明置信度头可以识别低价值后缀 token，并且这种修剪在聊天工作负载上最为显著，其中更高熵的 token 分布限制了固定长度验证的效率。在聊天子图中，提高阈值显著减少了被拒绝的 token，将接受率从 45.7% 提高到 95.7%。相比之下，结构化任务（数学和代码）经历较温和的修剪并保留更多草稿 token，接受率分别从 76.9% 上升到 92.5% 和从 67.6% 上升到 92.0%。

=== PAGE BREAK ===



每步平均 Token 数
数学



|---|---|---|---|---|

代码

| 0.0 | 0.2 | 0.4 | 0.6 | 0.8 |

|---|---|---|---|---|

聊天

| 76.9% | 92.5% |

| 67.6% | 92.0% |

| 40% | 60% | 80% | 100% |

接受率

| 45.7% | 95.7% |

置信度阈值

接受的 Token

被拒绝的 Token

接受率

图 5 | 置信度阈值扫描。阈值为 0
对应标准的固定长度验证。随着阈值增加，整体接受率稳步上升，因为置信度头有效修剪了最终会被拒绝的 token（斜线阴影柱状图）。

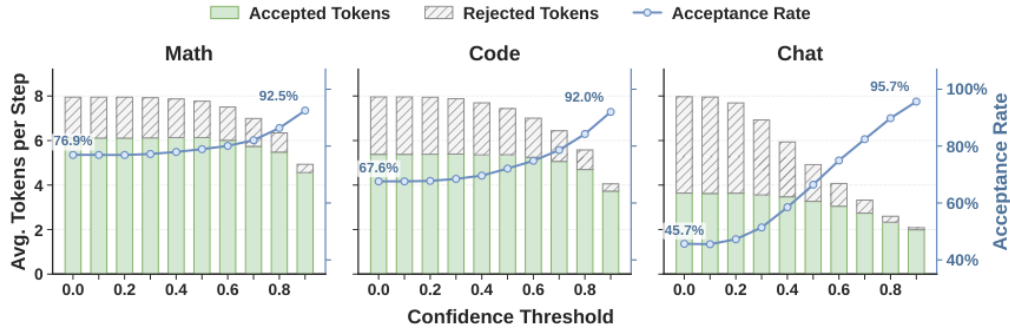


Figure 5 | **Confidence threshold sweep.** A threshold of 0 corresponds to standard fixed-length verification. As the threshold increases, the overall acceptance rate steadily rises because the confidence head effectively prunes tokens that would ultimately be rejected (hashed bars).

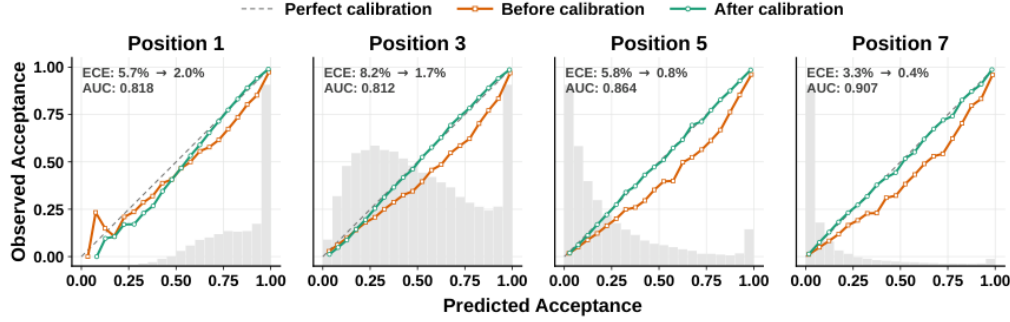


Figure 6 | **The Reliability Diagram on Alpaca Dataset.** While the raw confidence estimator achieves strong discrimination, its predictions are inherently overconfident. Applying post-hoc calibration helps to align the prefix survival probabilities with empirical acceptance rates. The shaded background histogram represents the frequency distribution of sample counts across different confidence bins.

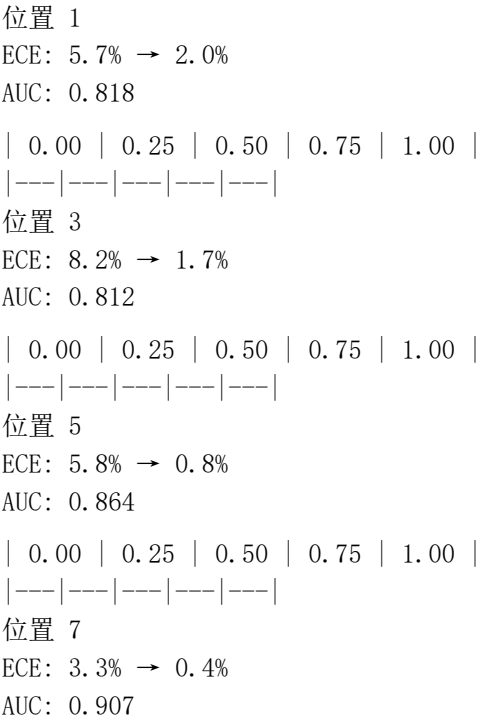
From Static Thresholds to Calibrated Scheduling. While useful for diagnostics, a static threshold is sub-optimal in dynamic serving environments because it ignores system load: verifying low-confidence tokens incurs minimal opportunity cost under low concurrency, but wastes critical batch capacity under high concurrency. This load dependency motivates the hardware-aware prefix scheduler. As formulated in [Section 3.2](#), maximizing system-level throughput requires the confidence model to exhibit both strong predictive discrimination and precise calibration to accurately estimate cumulative survival probabilities. The reliability diagram ([Figure 6](#)) demonstrates that while the raw model achieves strong discrimination (ROC-AUC ([Hanley and McNeil, 1982](#)) ranging from 0.81 to 0.90), it is overly confident (ECE 3%–8%). Applying post-hoc STS ([Section 3.2.1](#)) mitigates this overconfidence, reducing the average ECE to ~1% and yielding reliable survival estimates.

5. Real-World Deployment of DSpark

While [Section 4](#) establishes the algorithmic gains of DSpark on offline benchmarks, deploying it alongside large-scale models like DeepSeek-V4 ([DeepSeek-AI, 2026](#)) introduces additional

0.00	0.25	0.50	0.75	1.00
0.00	0.25	0.50	0.75	1.00

观察到的接受度



预测的接受度
完美校准
校准前
校准后

图 6 Alpaca 数据集上的可靠性图。虽然原始置信度估计器实现了很强的判别能力，但其预测天生过度自信。应用事后校准有助于将前缀存活概率与经验接受率对齐。阴影背景直方图表示不同置信度区间内样本计数的频率分布。

从静态阈值到校准调度。
虽然对诊断有用，但静态阈值在动态服务环境中是次优的，因为它忽略了系统负载：验证低置信度 token 在低并发下产生最小的机会成本，但在高并发下浪费关键的批处理容量。这种负载依赖性激发了硬件感知前缀调度器。如第 3.2 节所述，最大化系统级吞吐量要求置信度模型既具备强大的预测判别能力，又具备精确的校准能力，以准确估计累积存活概率。可靠性图（图 6）表明，虽然原始模型实现了强判别能力（ROC-AUC (Hanley and McNeil, 1982) 范围从 0.81 到 0.90），但它过度自信（ECE 3%–8%）。应用事后 STS（第 3.2.1 节）缓解了这种过度自信，将平均 ECE 降低到约 1%，并产生可靠的存活估计。

5. Real-World Deployment of DSpark

虽然第4节在离线基准测试上确立了 DSpark 的算法增益，但将其与 DeepSeek-V4 (DeepSeek-AI, 2026) 等大规模模型一起部署在训练和推理过程中引入了额外的系统级挑战。在本节中，我们介绍 DSpark 的端到端生产流水线。我们详细说明我们的可扩展训练机制、部署硬件感知前缀调度器（第3.2.2节）所必需的系统级优化，以及该框架在实时用户流量下的端到端性能。

5.1. Scalable and Flexible Training

DSpark 草稿模型与 DeepSeek-V4-Flash 和 DeepSeek-V4-Pro (DeepSeek-AI, 2026) 的预览版本共同部署。并行骨干网络包含三个 MoE 层 (Dai et al., 2024)，采用 mHC (Xie et al., 2026) 和滑动窗口注意力（窗口大小为128）。我们将最大块大小配置为 $\gamma = 5$ ，并利用马尔可夫头进行序列建模。此外，置信度头与草稿模型一起端到端训练，随后通过 STS 进行校准，以提供可靠的调度信号。

训练草稿模型需要目标模型的输出分布作为监督。对整个文档上下文评估两个模型会带来巨大的内存占用和工人间通信开销。为了解决这些瓶颈，我们在内部训练框架 (HAI-LLM) 4 中实现了两项系统级优化：

- 隐藏状态通信。将目标模型的全词汇 logits (V) $\approx$

105) 跨并行工人传输会产生显著的带宽瓶颈。我们改为临时缓存目标模型前向传播的激活值, 仅传输紧邻语言建模 (LM) 头的隐藏状态。然后, 仅在采样的目标位置上, 在草稿模型的工人上本地执行 LM 头投影。这将每 token 的通信复杂度降低到 $O(d)$, 其中 d 是隐藏维度。

•
锚定边界序列打包。为了将草稿模型的计算成本与目标模型的上下文长度解耦, 我们从训练序列中采样固定数量的草稿锚点, 并将这些孤立的预测块打包成密集的训练批次。我们通过 token 级别的注意力索引 (而非标准 2D 掩码) 管理这种打包。这保持了跨多个独立序列和锚点的精确因果掩码, 避免了与标准填充相关的计算和内存开销。

5.2. Hardware-Aware Prefix Scheduler in Practice

在第 3.2.2 节中, 算法 1 提供了一种理论上合理且无损的调度机制。然而, 将该算法直接部署到生产环境中会暴露出两个与实际基础设施的根本冲突。首先, 该算法假设一条平滑的单峰容量曲线, 而实际硬件容量 $SPS(B)$ 本质上是离散的, 表现出锯齿状的阶梯式退化 (Yan et al., 2020)。其次, 该算法要求每步调度动态的草稿 token, 这与连续的 CUDA Graph 重放 (Fireworks AI, 2023) 和零开销调度 (ZOS) (Zheng et al., 2024; Zhu et al., 2025) 相冲突。

为了在系统兼容性、吞吐量和算法正确性之间进行权衡, 我们调整调度器以异步方式运行。由于 ZOS 要求当前步骤完成之前就已知下一步的批大小, 同步调度将不可避免地停滞 GPU 流水线。我们改为使用两步之前的置信度头输出来估计即将到来的验证容量。在机制上, 当前步骤中的候选 token

仍然严格按照其实际的、最新的累积置信度得分排序; 来自两步之前的历史预测仅用于确定动态截断长度 (即批次容量限制 K)。这实际上将接纳过程转化为动态 top-K 选择。虽然近似容量 K 引入了轻微的时间偏移, 但选择机制在根本上是保持排序的: 最具置信度的草稿 token 总是优先进行验证。这种适配完全隐藏了调度延迟, 并确保与 ZOS 的无缝集成。

在此异步流水线的基础上, 我们解决了硬件利用率瓶颈。为了防止调度器因锯齿状的 SPS 悬崖而陷入局部最小值, 我们移除了早期停止的 break, 实现了无约束的全局搜索。通常情况下, 这种回溯性搜索会泄露未来 token 信息并违反无损保证 (附录 A)。然而, 我们的 ZOS 驱动适配自然防止了这一点。因为无约束搜索仅评估来自两步之前的历史预测, 接纳决策与当前 token $x_{r,k}$ 的实现是隔离的。截断长度本质上仅依赖于两步之前可用的信息。因此, 异步设计形成了一个因果屏障, 在跨硬件悬崖时最大化物理吞吐量, 同时保持精确的目标分布。

5.3. 高吞吐量与低延迟推理

5.3. High-Throughput and Low-Latency Inference

在解码过程中, 生产服务系统必须同时优化两个相互竞争的目标: 每请求延迟和总吞吐量 (Kwon et al., 2023; Zhao et al., 2025a; Zhong et al., 2024)。前者决定了单个用户的服务质量——这一因素在基于 Agent 的工作负载中日益关键 (Tiwari et al., 2026)——而后者决定了并发服务的用户总数。由于推测解码不可避免地会引入浪费的验证计算, 它本质上是在权衡这一取舍: 用额外的系统计算换取更快的每请求生成速度。

然而, 在我们的部署环境中, 每步处理的请求数量通常受到资源限制 (例如, 每个请求的固定 KV 缓存容量) 和可用用户流量池 (例如, RL 长尾负载) 的约束。因此, 有效批次大小始终远低于 GPU 的计算饱和阈值。在这种条件下, 传统的权衡被简化: 给定固定的并发限制, 最大化每 GPU 的总 token 吞吐量和最大化每用户的生成速度 (tok/s/user) 成为高度相关的目标, 而非相互竞争的目标。

为了实现这一最大吞吐量, 异步调度器 (第 5.2 节) 主动将空闲计算资源导向最有希望的草稿 token。然而, 执行这种动态路由在物理执行层引入了一个严峻的挑战: 推理框架必须高效地支持单个批次内的变长查询。标准的解码内核针对固定查询长度进行了高度优化; 朴素地处理变长的已验证前缀会因填充和不均匀的工作负载分布导致严重的 GPU 利用率不足。我们通过将物理执行与逻辑序列跟踪解耦来解决这个问题。在我们的计算内核中, 不同请求的所有 token 都被展开并作为独立元素以相同方式处理。然后, 复杂的序列内部依赖关系通过集成到我们稀疏注意力实现中的标记张量严格传递。具体在 DeepSeek-V4

架构上，只有索引注意力和压缩内核需要修改以支持这种变长路由，使得动态调度器能够无缝运行，而不引入底层执行开销。

Figure 7 | 吞吐量与 TPS 的对比。在实时流量下，总输出 token 吞吐量与每请求生成速度 (tok/s/user) 的关系。在我们的生产部署中，相对于 MTP-1 基线，DSpark 在实测流量和引擎配置下改善了观察到的吞吐量-交互性前沿。

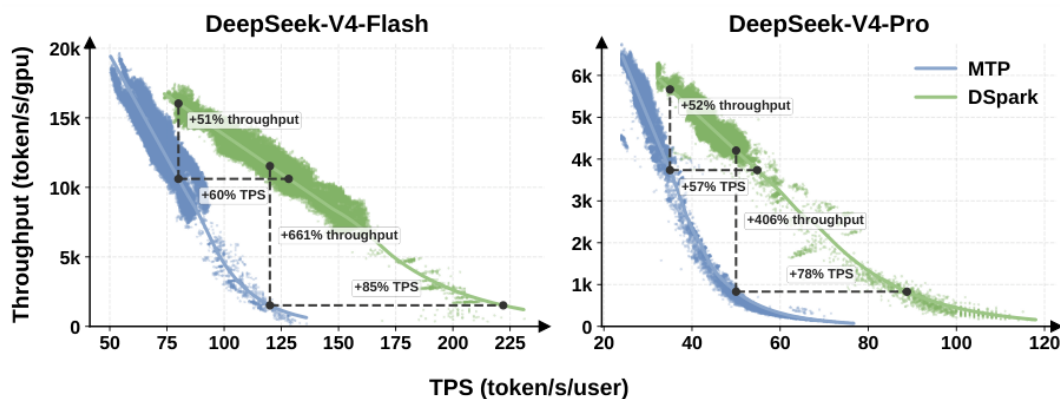


Figure 7 | **Throughput vs. TPS.** Aggregate output token throughput against per-request generation speed (tok/s/user) under live traffic. In our production deployment, DSpark improves the observed throughput–interactivity frontier relative to the MTP-1 baseline under the measured traffic and engine configurations.

5.4. Performance under Live User Traffic

We evaluate DSpark-5 (configured with a maximum draft length of $\gamma = 5$) against the MTP-1 (DeepSeek-AI, 2024) baseline within the production serving engines of DeepSeek-V4-Flash (preview) and DeepSeek-V4-Pro (preview). MTP-1 represents the former production setup, having been superseded by DSpark two weeks following the DeepSeek-V4-preview release. This single-token setup was historically maintained in production because deploying a static multi-token drafter (e.g., MTP-3/5) strictly degrades aggregate throughput under high concurrency due to excessive verification overhead. Therefore, comparing DSpark against this established baseline directly demonstrates its ability to safely unlock the performance potential of larger draft blocks in dynamic serving environments. In all figures, the scatter points represent raw telemetry data sampled directly from live user traffic, capturing complex, real-world request distributions, while the solid lines represent the fitted performance frontiers.

The Serving Pareto Frontier. Figure 7 illustrates the trade-off between aggregate system throughput and per-user generation speed (interactivity). To quantify DSpark’s behavior under practical deployment constraints, we evaluate the system at several interactivity SLA anchors. Here, an SLA (Service Level Agreement) specifies the minimum per-user generation speed (in tokens per second) that the system must guarantee.

For the V4-Flash engine, we evaluate the system at SLA anchors of 80 and 120 tok/s/user. At the moderate 80 tok/s/user SLA, DSpark improves aggregate throughput by 51% over the MTP-1 baseline. The stricter 120 tok/s/user SLA represents a qualitatively different regime: under this constraint, the single-token MTP-1 baseline approaches its operational boundary and can sustain only a very small concurrent batch. Consequently, the relative throughput ratio at this point is numerically large, with DSpark achieving a nominal 661% higher aggregate throughput. We therefore interpret this high-SLA point primarily as evidence that DSpark extends the feasible interactivity frontier, rather than as a representative multiplicative speedup over a well-utilized baseline. At matched practical throughput levels, which provide a more stable comparison, DSpark accelerates per-user generation speeds by 60% to 85%.

5.4. Performance under Live User Traffic

我们评估了 DSpark-5（配置最大草稿长度 $\gamma = 5$ ）与生产服务引擎中 DeepSeek-V4-Flash（预览版）和 DeepSeek-V4-Pro（预览版）上的 MTP-1（DeepSeek-AI, 2024）基线。MTP-1 代表先前的生产配置，在

DeepSeek-V4-preview 发布两周后被 DSpark 取代。这种单 token 配置历史上一直维护在生产环境中，因为部署静态的多 token 起草器（例如 MTP-3/5）在高并发下由于过多的验证开销会严格降低总吞吐量。因此，将 DSpark 与此已建立的基线直接比较，证明了其在动态服务环境中安全解锁更大草稿块性能潜力的能力。在所有图中，散点代表直接从实时用户流量中采样的原始遥测数据，捕捉了复杂的真实世界请求分布，而实线表示拟合的性能前沿。

服务帕累托前沿。

The Serving Pareto Frontier.

图7展示了系统总吞吐量与每用户生成速度（交互性）之间的权衡。为了量化 DSpark 在实际部署约束下的行为，我们在几个交互性 SLA 锚点评估了系统。这里，SLA（服务水平协议）指定了系统必须保证的最小每用户生成速度（以每秒 token 数计）。

对于 V4-Flash 引擎，我们在80和120 tok/s/user 的 SLA 锚点评估系统。在适中的80 tok/s/user SLA 下，DSpark 的总吞吐量比 MTP-1 基线提高了51%。更严格的120 tok/s/user SLA 代表了一种性质不同的场景：在此约束下，单 token 的 MTP-1 基线接近其操作边界，只能维持非常小的并发批次。因此，此点的相对吞吐量比率在数值上很大，DSpark 实现了标称的661% 更高的总吞吐量。因此，我们将这个高 SLA 点主要解释为 DSpark 扩展了可行交互性前沿的证据，而非相对于一个充分利用的基线的代表性倍数加速。在匹配的实际吞吐量水平（提供更稳定的比较）下，DSpark 将每用户生成速度加速了60%到85%。

V4-Pro 部署显示出相同的模式。在适中的35 tok/s/user SLA 下，DSpark 的总吞吐量提高了52%。在更严格的50 tok/s/user SLA 下，MTP-1 再次进入低并发区域，为 DSpark 带来标称的406% 相对吞吐量优势。与 V4-Flash 一样，我们将此点视为 DSpark 在基线无法有效支持的交互性目标下维持有用吞吐量的迹象。在匹配的系统容量下，DSpark 提供了57%到78% 更快的每用户生成速度。总体而言，这些结果表明 DSpark 将观察到的吞吐量-交互性前沿向外推移：它在中等 SLA 条件下改善了吞吐量，更重要的是，在严格的交互性约束下保持了非退化的服务容量。

图8 | 负载自适应吞吐量和验证预算。顶行（a, b）：不同系统并发水平下的总输出吞吐量。底行（c, d）：每请求分配的平均目标验证预算。随着并发负载的增加，动态调度器自动限制每请求验证长度以防止资源争用。

The V4-Pro deployment shows the same pattern. At the moderate 35 tok/s/user SLA, DSpark improves aggregate throughput by 52%. At the stricter 50 tok/s/user SLA, MTP-1 again enters a low-concurrency regime, yielding a nominal 406% relative throughput advantage for DSpark. As with V4-Flash, we treat this point as an indication that DSpark sustains useful throughput under an interactivity target that the baseline cannot efficiently support. At matched system capacities, DSpark delivers 57% to 78% faster per-user generation. Overall, these results show that DSpark shifts the observed throughput–interactivity frontier outward: it improves throughput in moderate-SLA regimes and, more importantly, preserves non-degenerate serving capacity under strict interactivity constraints.

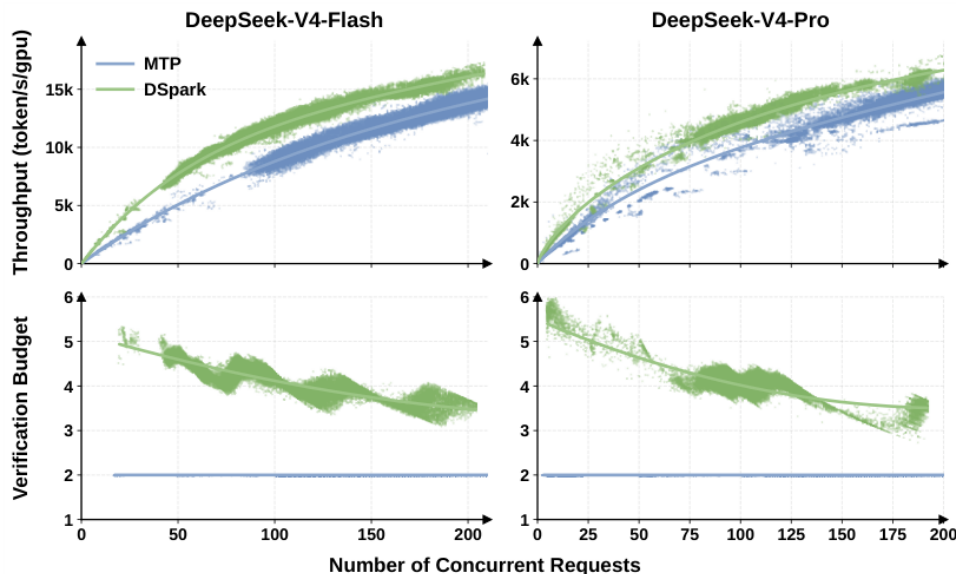


Figure 8 | **Load-adaptive throughput and verification budgets.** Top row (a, b): Aggregate output throughput across varying levels of system concurrency. Bottom row (c, d): The average target verification budget allocated per request. As concurrent load increases, the dynamic scheduler automatically restricts the per-request verification length to prevent resource contention.

Throughput Dynamics under Load. Figure 8 analyzes the underlying mechanism driving these gains by plotting aggregate throughput (top row) and the dynamic verification budget (bottom row) against system concurrency.

- Under the moderate concurrency regimes typical of our production deployment (fewer than 200 concurrent requests for V4-Flash and 150 for V4-Pro), the hardware-aware scheduler leverages available target compute capacity by allocating longer verification budgets, expanding from MTP-1’s static 2 tokens to roughly 4–6 tokens per request. This extended verification yields more accepted tokens per forward pass, directly contributing to the throughput gains observed on the Pareto frontier.
- As system concurrency scales and target capacity saturates, the scheduler dynamically restricts this budget. The average verification length decreases smoothly with load, ensuring that low-confidence draft tokens are pruned before they consume critical batch capacity. This load-aware behavior stabilizes production deployment: DSpark maximizes

负载下的吞吐量动态。

Throughput Dynamics under Load.

图8通过绘制总吞吐量（顶行）和动态验证预算（底行）与系统并发度的关系，分析了驱动这些增益的底层机制。

- 在我们的生产部署中典型的适中并发条件下（V4-Flash 少于200个并发请求，V4-Pro 少于150个），硬件感知调度器通过分配更长的验证预算来利用可用的目标计算能力，从 MTP-1 的静态2个 token 扩展到每请求约4-6个 token。这种扩展的验证使得每次前向传播获得更多被接受的 token，直接促成了帕累托前沿上观察到的吞吐量增益。

- 随着系统并发度增加且目标容量饱和，调度器动态地限制这一预算。平均验证长度随负载平滑下降，确保低置信度的草稿 token 在消耗关键批次容量之前被剪除。这种负载感知行为稳定了生产部署：DSpark 在轻负载下最大化空闲计算的效用，同时在重负载下有效保护关键批次容量。

局限性。

Limitations.

尽管前缀调度器最小化了目标模型验证的浪费，DSpark

仍然需要固定的草稿端成本来通过并行骨干网络生成初始的

γ -token

块。对于接受率本就真的很低的复杂查询，这种前期的起草计算是无法回收的。未来的优化可以在草稿模型中引入难度感知的提前退出机制，使此类请求能够跳过完整块的生成。

6. Related Work

推测解码算法。

Speculative Decoding Algorithms.

推测解码通过将 token 提议与验证解耦来加速自回归生成。在早期块状方法 (Ge et al., 2022; Stern et al., 2018; Sun et al., 2021; Xia et al., 2023) 的基础上，现代方法采用拒绝采样来精确保持目标模型的分布 (Chen et al., 2023; Leviathan et al., 2023)。由于推理加速直接取决于起草器的效率和准确性，大量研究集中于优化其架构。除了使用独立的小语言模型 (Chen et al., 2023; Leviathan et al., 2023)，后续工作将多 token 头或特征外推器直接集成到目标模型中 (Ankner et al., 2024; Cai et al., 2024, 2025; DeepSeek-AI, 2024; Gloeckle et al., 2024; Li et al., 2024b,c, 2026b; Zhang et al., 2025)。其他策略包括通过提前退出进行自我推测 (Elhoushi et al., 2024; Liu et al., 2024a; Xia et al., 2025; Zhang et al., 2024)、动态词汇压缩 (Williams et al., 2026; Zhao et al., 2025b)、提示查找 (Saxena, 2023; Somasundaram et al., 2025)、后缀自动机 (Hu et al., 2025) 和检索 (He et al., 2023; Shen et al., 2026)。为了消除起草本身的顺序瓶颈，最近的方法提出了并行或块状生成。P-EAGLE 并行化了 EAGLE 风格的起草 (Hui et al., 2026)，而 PARD、DART 和 DFlash 使用扩散启发的预测在单次前向传播中生成整个块 (An et al., 2026; Chen et al., 2026; Liu et al., 2026a)，DDTree 随后将其扩展到可验证的草稿树 (Ringel and Romano, 2026)。并行工作也改进了 DFlash: Domino (Huang et al., 2026a) 引入了一个与我们的 RNN Head 概念上相似的 CausalEncoder，而 DFlare (Zhang et al., 2026a) 通过逐层融合解决了条件瓶颈。

推测解码的系统感知调度。

System-Aware Scheduling for Speculative Decoding.

除了起草器架构之外，另一条工作线专注于确定每轮生成或验证的最佳推测 token 数量。为此，各种方法使用置信度启发式 (Du et al., 2024; Li et al., 2024b; Liu et al., 2026c; Mamou et al., 2024; Wen and Feng, 2026)、学习到的接受预测器 (Huang et al., 2024; Zacks917, 2026) 或赌博机风格的策略 (Liu et al., 2026b) 来动态调整草稿长度。此外，认识到推测解码本质上是一个系统级调度问题，最近的研究通过根据实时系统负载和请求优先级调整推测预算来优化总体吞吐量和延迟 (AngelSlim Team, 2026; Hu et al., 2026; Huang et al., 2026b; Li et al., 2026a; Liu et al., 2024c; Miao et al., 2024; Sadhukhan et al., 2025; Wu et al., 2025)。

并行生成。

Parallel Generation.

并行生成

token

的模型提供了几乎与输出长度无关的解码延迟，使其成为自回归解码的一个有吸引力的替代方案。非自回归 Transformer (NATs, Gu et al., 2018) 开创了这一方向，通过在单次前向传播中独立预测所有位置。然而，这迫使模型在所有可能的模式上

取平均，往往产生混合了来自不同有效序列片段的输出。已经出现了两条主要工作线来解决这一局限性。一条方向保留了单次前向传播的架构，但改变了模型所见的内容或其训练方式：引入潜在变量作为条件输入，将所有位置引导向一致的输出（Gu et al., 2018; Kaiser et al., 2018; Ma et al., 2019），或者放宽训练目标，使模型专注于生成一个单一的连贯输出，而不是对所有有效替代方案的全分布进行建模（Du et al., 2021; Qian et al., 2021; Shao et al., 2021, 2023）。另一条方向通过迭代重新预测（Austin et al., 2021a; Ghazvininejad et al., 2019; Li et al., 2022）、块级自回归（Arriola et al., 2025; Wang et al., 2018）或结构化输出层（如 CRF（Sun et al., 2019）、CTC（Libovický and Helcl, 2018; Saharia et al., 2020）、HMM（Huang et al., 2022b）和 PCFG（Gui et al., 2023））重新引入了有限的序列依赖性。

推测解码进一步要求起草器必须为拒绝采样规则提供精确的每 token 概率。由于迭代精化、潜在变量边缘化或全局归一化，上述大多数技术无法轻易提供这样的概率。例如，在我们密切相关的设计 CRF-NAT（Sun et al., 2019）中，也在并行隐藏状态之上放置了一个序列模块，但其全局归一化的配分函数阻止了精确的每 token 概率计算。类似地，在将 CTC 输出层适配到并行推测解码时，CTC-drafter（Wen et al., 2024）由于对齐路径的潜在变量边缘化而受到贪心验证的限制。DSpark 通过将序列校正保持为局部操作来回避这些限制，因此每 token 概率仍然是精确的 softmax 评估。

7. Conclusion

在本文中，我们提出了 DSpark，一个旨在克服大语言模型在高并发生产环境中的结构和系统级瓶颈的推测解码框架。在算法层面，DSpark 引入了一种半自回归生成范式——将计算密集型的并行骨干网络与轻量级的序列头相结合——以缓解独立并行起草器的快速后缀衰减。在系统层面，我们将验证长度选择形式化为一个全局吞吐量最大化问题，采用一个硬件感知前缀调度器，该调度器基于校准的生存概率和实时引擎负载动态调整目标模型的验证预算。广泛的离线评估表明，DSpark 在多种领域中显著优于最先进的自回归和并行基线。此外，其在 DeepSeek-V4 中的实际部署验证了其在生产服务中的实用价值：通过智能管理验证开销，DSpark 在重负载下维持了稳健的并发能力，持续加速了每用户生成速度，并有效地将 LLM 服务的帕累托前沿向外推移。